

Kafka UI: Genel Kullanım ve Eklediğimiz Özellikler

Kafbat UI (açık kaynak, Apache-2.0) tabanlı Kafka arayüzümüz: uygulama ne işe yarar, nasıl kullanılır ve fork ile neler ekledik.

03.07.2026 | 20 milyon mesajlık gerçek test ortamında canlı doğrulandı

Bu doküman

Üç bölümden oluşur. Önce uygulamanın kendisi: Kafka UI nedir, günlük işte hangi ekran ne için kullanılır. Sonra kısa bir "perde arkası": araç işleri nasıl hallediyor. En sonda fork ile eklediğimiz üç özellik. Eklenenlerin özeti:

Özellik	Ne işe yarıyor
1. Takılmayan arama	Orijinal araçta sonuçsuz mesaj araması, topic'in tamamını tarayıp dakikalarca kilitleniyordu. Artık arama bir "tarama bütçesi" ile saniyeler içinde duruyor, istenirse kaldığı yerden devam ediyor.
2. Grafik butonu	Topic sayfasındaki tek bir butonla, mesajların içindeki JSON alanlarından anında grafik çizen bir araç açılıyor (dağılım, zaman serisi, toplam, ortalama). Örneklem, temsili örneklem ve tüm veriyi kesin sayma modlarıyla. Piyasadaki ücretsiz araçların hiçbirinde bu yok.
3. Tek adres, tek komut	Her şey tek bir <code>docker compose up -d</code> ile ayağa kalkıyor ve tek porttan (80) sunuluyor: ana arayüz <code>/</code> , grafik aracı <code>/viz/</code> .

Uygulama nedir, genel olarak nasıl kullanılır?

Kafka UI, Kafka'yı komut satırı kullanmadan tarayıcıdan izlemeye ve yönetmeye yarayan bir web arayüzüdür. Kafka'nın kendi ekranı yoktur; içinde ne aktığını görmek normalde terminal komutları gerektirir. Bu araç o ihtiyacı tek adresten karşılar: topic'lere bakmak, mesajların içeriğini okumak, test mesajı göndermek, tüketicilerin nerede kaldığını izlemek.

Sol menü dört ana bölümden oluşur: **Dashboard** (bağlı cluster'ların listesi ve sağlık özeti), **Brokers** (sunucu sağlığı), **Topics** (mesaj kanalları, asıl çalışma alanı) ve **Consumers** (tüketici grupları ve lag takibi).

Topics: mesaj kanallarının listesi

Günlük kullanımın başladığı yer. Tüm topic'ler partition sayısı, mesaj adedi ve disk boyutuyla listelenir. Buradan:

- Yeni topic oluşturulur:** sağ üstte "Add a Topic" (ad, partition sayısı, saklama süresi).
- Topic aranır:** üstteki kutuya adın bir parçasını yazmak yeter.
- Toplu işlem yapılır:** seçilen topic'ler silinebilir veya içindeki mesajlar temizlenebilir (purge).

kafbat UI

5c26356 03.07.2026 08:53:06

UTC+3:00

Dashboard

demo

Brokers

Topics

Consumers

demo

Topics

+ Add a Topic

Export CSV

Show Internal Topics

Delete selected topics

Copy selected topic

Purge messages of selected topics

☐	Topic Name	Partitions	Out of sync replicas	Replication Factor	Number of messages	Size	
☐	test-20m	6	0	1	20000002	2 GB	

Topics ekranı: topic listesi, mesaj sayıları ve boyut; sağ üstte yeni topic oluşturma.

Mesajları okumak: Topic > Messages

Bir topic'e tıklayıp Messages sekmesine geçince en yeni mesajlar listelenir. Bir satıra tıklayınca mesajın içeriği renklendirilmiş JSON olarak açılır; Key / Value / Headers sekmeleriyle mesajın her parçası incelenir. Üstteki araç çubuğundan:

- Yön seçilir:** Newest (en yeniden geriye) veya Oldest (en baştan ileriye), belirli tarihten de mümkün.
- Partition daraltılır:** sadece belirli partition'lara bakılabilir.
- Aranır:** sağdaki kutuya yazılan metin mesaj içeriğinde ve key'de aranır (fork ile artık güvenle, bkz. Özellik 1). Daha karmaşık şartlar için "Add Filters" ile alan bazlı filtre yazılabilir.

Dashboard

demo

Brokers

Topics

Consumers

demo

Topics / test-20m

Produce Message

Grafik

Overview

Messages

Consumers

Settings

Statistics

ACLs

Newest

Select partitions

Key Serde

Value Serde

Refresh

Search

5 ms

28 KB

102 messages consumed

Offset	Partition	Timestamp	Key Preview	Value Preview
3369735	4	02.07.2026 19:34:39.429		{"event_id":"NEEDLE-002","event_type":"needle_t..
3369734	4	02.07.2026 19:34:39.426		{"event_id":"NEEDLE-001","event_type":"needle_t...
3435497	5	02.07.2026 19:33:59.875		{"event_id":"evt-005313","event_type":"order_deli...

Key

Value

Headers

```
{
  "event_id": "evt-005313",
  "event_type": "order_delivered",
  "user": {
    "id": 9431,
    "name": "burak",
    "city": "istanbul"
  },
  "order": {
    "id": "ord-657842",
    "amount": 1587.21,
    "currency": "TRY",
    "items": 9
  },
  "ts": 1753732085,
  "source": "web",
  "meta": {
    "session": "s-1604740465",

```

Timestamp

02.07.2026 19:33:59.875

Timestamp type: CREATE_TIME

Key Serde

Size: 0 Bytes

Value Serde

String

Size: 280 Bytes

Mesajlar ekranı: mesaj listesi ve tıklanan mesajın açılmış JSON içeriği (Key / Value / Headers).

Mesaj göndermek: Produce Message

Topic sayfasının sağ üstündeki Produce Message butonu, elle mesaj göndermek içindir: key, value ve header girilir, istenirse partition seçilir. En sık kullanımı: bir tüketici uygulamayı hızlıca test etmek veya bir hata senaryosunu yeniden üretmek.

Consumers: kim nereye kadar okudu? (lag takibi)

Kafka'da her tüketici uygulama bir "consumer group" ile okur ve kaldığı yer (offset) kaydedilir. Bu ekran her grubun hangi topic'i nereye kadar okuduğunu ve lag değerini gösterir. Lag, yazılmış ama o grup tarafından henüz okunmamış mesaj sayısıdır: sıfıra yakınsa tüketici yetişiyor, sürekli büyüyorsa uygulama geride kalıyor veya durmuş demektir. Üretimde en çok bakılacak ekran budur.

kafbat UI

5c26356 03.07.2026 08:53:06

UTC+3:00

Dashboard

demo

Brokers

Topics

Consumers

demo

Consumers

Search by Consumer Group ID

Export CSV

Refresh rate: Off

Group ID	Num Of Members	Num Of Topics	Consumer Lag	Coordinator	State
demo-izleyici	0	1	19700002	1	EMPTY

Consumers ekranı: demo-izleyici grubu 300 bin mesaj okuyup durdu, kalan 19,7 milyon mesaj lag olarak görünüyor.

Brokers: sunucu sağlığı

Kafka sunucularının (broker) durumu tek bakışta: kaç broker ayakta, disk kullanımı, partition'ların sağlığı (online / in-sync / out-of-sync). Bir şeylerin ters gittiğinden şüphelenildiğinde ilk bakılacak yer.

kafbat UI

5c26356 03.07.2026 08:53:06

UTC+3:00

Dashboard

demo

Brokers

Topics

Consumers

demo

Brokers

Export CSV

Uptime

Broker Count

1

Active Controller

1

Version

3.9-IV0

Partitions

Online

6 of 6

URP

0

In Sync Replicas

6 of 6

Out Of Sync Replicas

0

Controller Type

KRaft

Broker ID

Disk usage

In Sync Replicas

Replicas

Replicas skew

Leaders

Leaders skew

Port

1

1.76 GB, 6 segment(s)

6

6

-

6

-

9092

Brokers ekranı: broker sayısı, sürüm, disk kullanımı ve partition sağlığı.

Topic detayındaki diğer sekmeler

Sekme	Ne gösterir / ne yapılır
Overview	Partition dağılımı, offset'ler, toplam mesaj sayısı, saklama politikası.
Consumers	Sadece bu topic'i okuyan gruplar ve lag'leri.
Settings	Topic ayarları (saklama süresi, boyut limitleri vb.), düzenlenebilir.
Statistics	İstenirse topic taranıp boyut/anahtar istatistikleri çıkarılır.

Perde arkası: araç işleri nasıl hallediyor?

Teknik olmayan bir özet: bu araç Kafka'nın önüne konmuş canlı bir **vitrin ve kumanda**dır. Verinizi kendi içine taşımaz, çoğaltmaz; işin ağırlığı hep Kafka'da kalır, araç yalnızca sorar ve gösterir. Ekrandan bir istek geçtiğinde izlediği yol:



"Grafik" butonu bu zincirin yanında ayrı bir yol açar: Tarayıcı → nginx (/viz/) → kafka-viz → Kafka. Ana araç sade kalsın diye grafik işini ayrı bir servis yapar.

Soru	Kısa cevap
Verimi kendine kopyalıyor mu?	Hayır. Mesajlarınızın kopyasını tutmaz; her ekranı açtığınızda Kafka'ya o an sorar ve gelen cevabı gösterir. Yalnızca pano ve metrik gibi özet bilgileri, kısa aralıklarla (varsayılan her 30 saniyede bir) yenilenen bir özet olarak tutar. Gördüğünüz mesajlar her zaman canlı Kafka'dır, eski bir kopya değil.
20 milyon mesajı nasıl anında açıyor?	Hepsini okumaz. Kafka'nın kendi okuma mekanizmasıyla doğru noktaya atlar (offset), bir ekranlık mesaj okur ve gösterir. "Daha eski" dedikçe bir sonraki sayfayı ister. Bu yüzden topic ne kadar büyük olursa olsun açılış hızlıdır.
Mesaj sayısı ve "lag" nereden geliyor?	Kafka'nın kendi kayıtlarından (offset'ler). Her tüketici grubunun nereye kadar okuduğu Kafka'da yazılıdır; araç bunu okuyup grubun ne kadar geride kaldığını (lag) hesaplar. Kendi sayacını tutmaz.
Broker metrikleri nereden? (Metrics sekmesi)	Kafka sürecinin içindeki canlı sağlık sayaçlarından (JMX). Araç bu sayaçları okuyup gösterir. Bu sürümde JMX'i açtık; artık Metrics sekmesi gerçek broker/JVM metriklerini getiriyor.
Topic açma, mesaj gönderme gibi işler?	Araç bir uzaktan kumanda gibi çalışır: bu komutları Kafka'nın yönetim arayüzüne iletir, işi Kafka yapar.
Yanlışlıkla bir şeyi bozar mı?	Normal kullanımda hayır, sadece bakar (salt okur). Değiştiren işlemler (silme, temizleme, ayar değiştirme) ayrı ve belirgin butonlardadır; kendi kendine tetiklenmez.

Özetle: Kafka UI hafif bir arayüzdür çünkü asıl veriyi tutan, saklayan ve hesaplayan hep Kafka'dır. Araç veriyi taşımadan, önüne canlı bir pencere koyar. Bu tasarım büyük topic'lerde bile hızlı kalmasını ve tek başına bir "veri kopyası" riski oluşturmamasını sağlar.

Fork ile eklediğimiz özellikler

Buraya kadar anlatılanlar aracın hazır gelen yetenekleri. Aşağıdaki üç bölüm ise bizim eklediklerimiz.

1. Takılmayan arama (tarama bütçesi)

Sorun neydi?

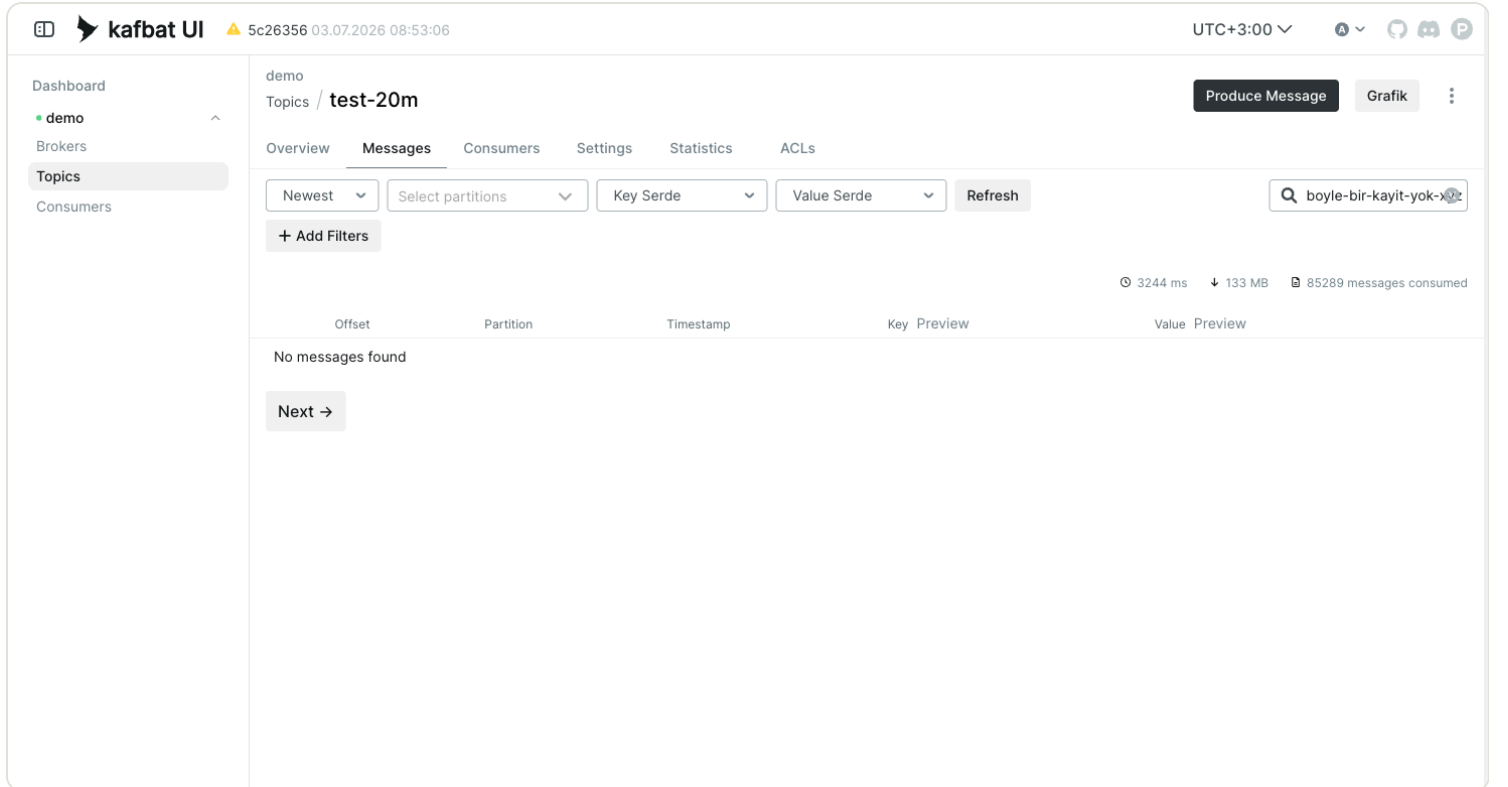
Kafbat UI'da bir topic içinde mesaj ararken aradığınız metin yoksa, araç eşleşme bulana kadar topic'in tamamını taramaya devam ediyor. 20 milyon mesajlık bir topic'te bu, dakikalarca dönen ve durdurulamayan bir ekran demek. Bu, projenin kendi kayıtlarında da bilinen bir hata (upstream issue #442) ama henüz çözülmemiştir.

Ne yaptık?

Aramaya bir tarama bütçesi ekledik: arama en fazla 500.000 kayıt tarar veya 30 saniye çalışır, hangisi önce dolarsa orada durur. Durduğunda o ana kadar bulunanlar ekrandadır ve "Next" butonuyla kaldığı yerden bir sonraki dilimi taratabilirsiniz. Yani kontrol artık kullanıcıda: küçük dilimler halinde istediğiniz kadar derine inersiniz, araç asla kilitlenmez.

Sonuç (canlı test)

20 milyon mesajlık topic'te hiç eşleşmeyen bir arama: orijinalde süresiz takılıyordu, fork'ta 3,2 saniyede durdu (aşağıda sağ üstte 3244 ms, 85.289 mesaj tarandı; sol altta devam için Next butonu).



Sonuçsuz arama: 3,2 saniyede durdu, "No messages found" + Next ile devam seçeneği.

İnce ayar (opsiyonel): Varsayılanlar çoğu durum için yeterli. Gerekirse `kafka.polling.maxScannedRecords` (varsayılan 500.000) ve `kafka.polling.responseTimeoutMs` (varsayılan 30.000 ms). Docker ortam değişkeni olarak: `KAFKA_POLLING_MAXSCANNEDRECORDS`, `KAFKA_POLLING_RESPONSETIMEOUTMS`.

2. "Grafik" butonu ve kafka-viz

Sorun neydi?

Pazar arařtırmamızda gördük: ücretsiz Kafka arayüzlerinin hiçbirisi mesajların içeriğinden grafik üretemiyor. "Bu topic'te hangi event tipi ne kadar var?", "dakikada kaç mesaj akıyor?" gibi sorular için veriyi dışarı çekip elle analiz etmek gerekiyordu.

Ne yaptık?

İki parça:

- **kafka-viz:** Ayrı, hafif bir grafik aracı. Topic'in JSON alanlarını otomatik keşfeder (iç içe alanlar dahil, örn. `user.city`) ve seçilen alan için grafik çizer. Veriyi nasıl okuduğu seçilebilir (aşağıdaki örnekleme modları).
- **"Grafik" butonu:** Kafka UI'da her topic sayfasının sağ üstüne eklendi. Basınca kafka-viz yeni sekmede, o topic seçili olarak açılır. Grafik motoru Kafka UI'ın içine gömülmedi; ana araç sade kaldı, grafik aracı bağımsız gelişebiliyor.

kafbat UI

5c26356 03.07.2026 08:53:06

UTC+3:00

Dashboard

demo

Brokers

Topics

Consumers

demo

Topics / test-20m

Produce Message

Grafik

Overview

Messages

Consumers

Settings

Statistics

ACLs

Partitions

6

Replication Factor

1

URP

0

In Sync Replicas

6 of 6

Type

External

Segment Size

2 GB

Segment Count

6

Clean Up Policy

DELETE

Message Count

20000002

Partition ID	Replicas	First Offset	Next Offset	Message Count
0	1	0	3335487	3335487
1	1	0	3200340	3200340
2	1	0	3354479	3354479
3	1	0	3304462	3304462
4	1	0	3369736	3369736
5	1	0	3435498	3435498

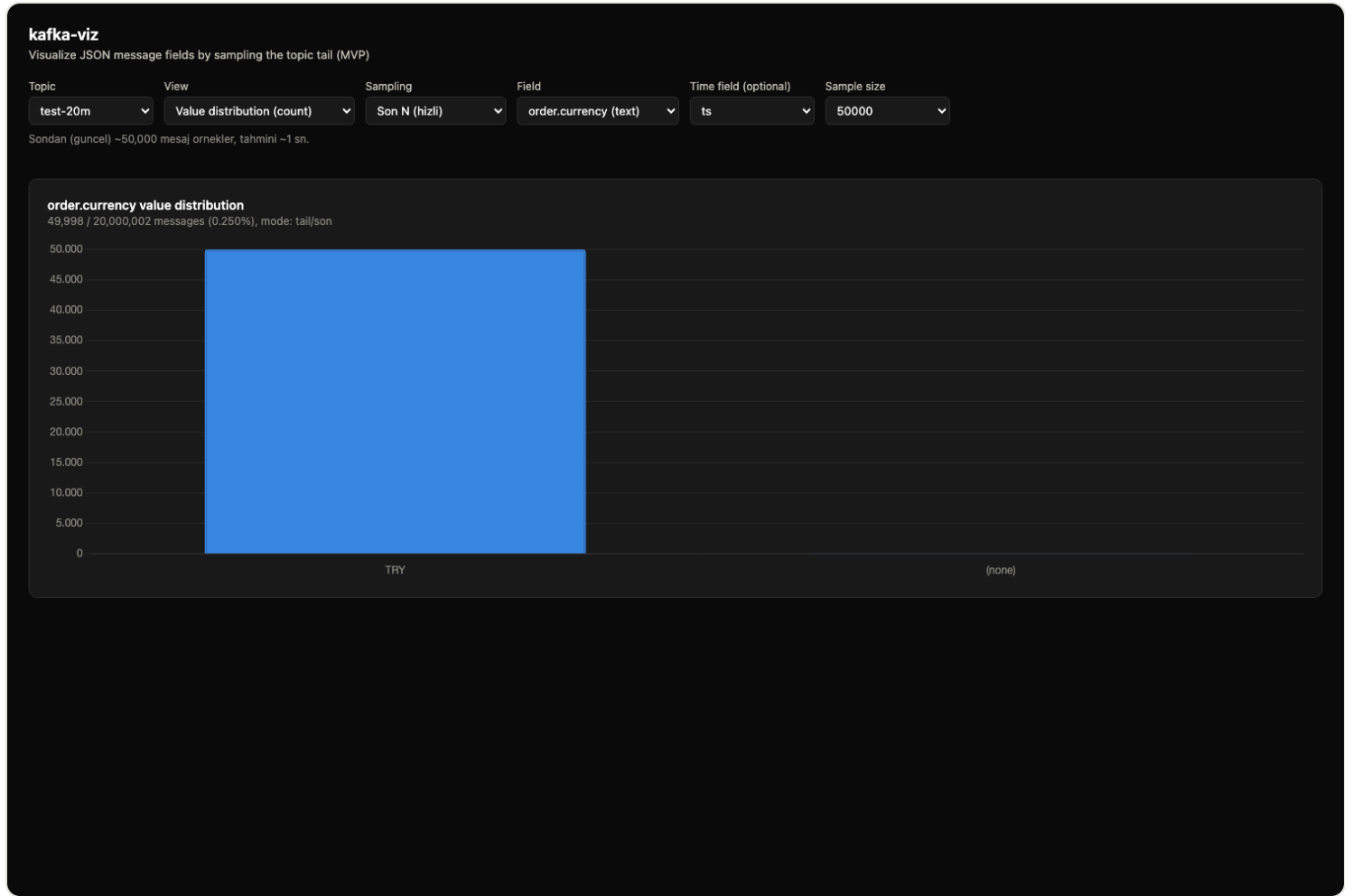
Topic sayfası: sağ üstte "Produce Message" yanına eklenen "Grafik" butonu (topic: test-20m, 20.000.002 mesaj).

Örnekleme modları YENİ

20 milyonluk bir topic'i her açılıřta baştan sona okumak yavaş olur. Bunun yerine kafka-viz üç mod sunar; hangi soruyu sorduğunuza göre hız ile kesinlik arasında seçim yaparsınız:

Mod	Ne yapar	Ne zaman
Son N (hızlı)	Topic'in sonundan (en güncel) örnek alır. Saniyeler içinde sonuç.	Şu anki duruma hızlı bakış.
Yayılmış (temsili)	Örneği topic'in başından sonuna eşit aralıklarla alır; dağılım tüm veriyi temsil eder. Çok sayıda konumdan okuduğu için Son N'den yavaştır (bu topic'te onlarca saniye).	Genel dağılımın doğru olması gerektiğinde.
Tümü (kesin)	Tüm mesajları okuyup kesin sayar (akış halinde, bellek şişmez). Yavaş.	Kesin rakam gerektiğinde (yalnız değer dağılımı).

Örnek boyutu 5 bin ile 1 milyon arasında seçilir. Her grafiğin altında artık **gördüğünüz mesaj / toplam mesaj (yüzde kapsam)** ve kullanılan mod yazar; hangi kontrolde ne kadar süreceğine dair bir tahmin ipucu da gösterilir.



kafka-viz: test-20m topic'inde değer dağılımı. Üstte "Sampling" mod seçici; grafik altında "49.998 / 20.000.002 (%0,25), mod: tail" kapsam bilgisi.

Görünümler

Görünüm	Cevapladığı soru	Örnek
Değer dağılımı	Hangi değerden kaç adet var?	event_type'a göre mesaj sayıları
Zaman serisi	Dakikada kaç mesaj akıyor?	Trafik yoğunluğu, ani düşüş/artış
Zaman içinde toplam	Sayısal bir alanın toplamı nasıl seyrediyor?	order.amount ile dakikalık ciro
Zaman içinde ortalama	Sayısal bir alanın ortalaması nasıl seyrediyor?	Ortalama sepet tutarı

Araç, seçilen görünüme uymayan alanları otomatik gizler (örn. "toplam/ortalama" seçince yalnız sayısal alanlar listelenir) ve varsa bir zaman alanını kendiliğinden seçer; böylece boş veya hatalı grafik oluşmaz. Bozuk veya JSON olmayan mesajlar aracı çökertmez; atlanır ve atlanan sayısı raporlanır. Araç topic'e hiçbir şey yazmaz, salt okurdur.

Uçtan uca test (Playwright) YENİ

test-20m topic'inde (20.000.002 mesaj) her mod gerçek bir tarayıcıda otomatik test edildi (Playwright, izole Chromium). Yedi senaryonun tamamı geçti. Ölçülen süreler:

Senaryo	Sonuç	Süre
Son N (tail), event_type	Grafik güncellendi, %0,25 kapsam	~0,6 sn
Yayılmış (spread), event_type	Grafik güncellendi, baştan sona temsili	~34 sn
Tümü (full), event_type	20.000.002 / 20.000.002 (%100), kayıp yok	~115 sn
Tümü: tarama sırasında UI	Görünüm ve örnek kilitli, grafik soluk, "taranıyor" yazısı	anında
Tümü: alan değişince	Eski "%100" kalmıyor; grafik soluklaşıp "taranıyor" diyor, sonra doğru alana güncelliyor	~84 sn
Tarama sürerken alan keşfi (/api/fields)	Bloke olmuyor, hızlı cevap	~0,3 sn

kafka-viz

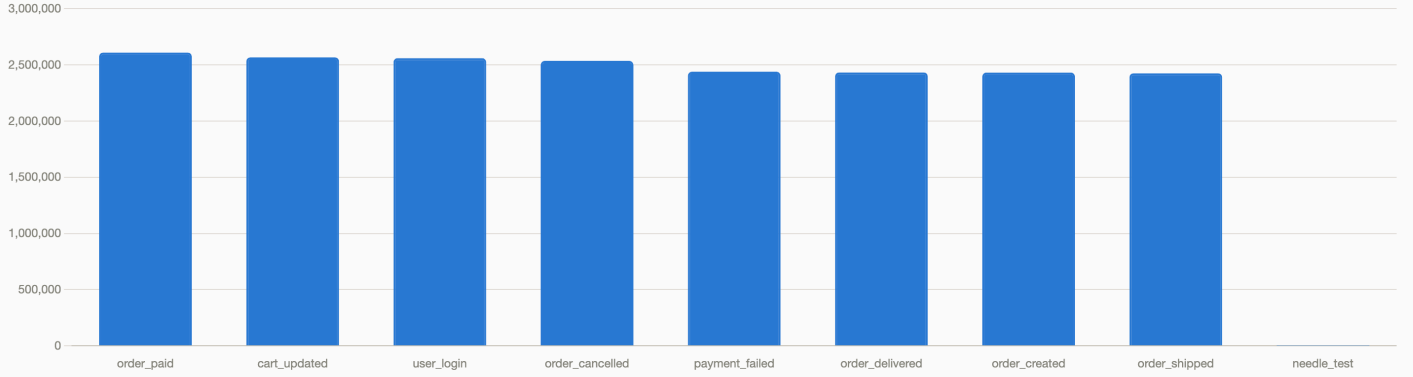
Visualize JSON message fields by sampling the topic tail (MVP)

Topic: test-20m View: Value distribution (count) Sampling: Tümü (kesin, yavas) Field: event_type (text) Time field (optional): ts Sample size: 50000

Tüm veriyi tarayıp kesin sayar; büyük topic'te dakikalar sürebilir. Yalnız değer dağılımı.

event_type value distribution

20,000,002 / 20,000,002 messages (100.0%), mode: full/tüm



"Tümü" modu bitti: `event_type` için 20 milyonun tamamı kesin sayıldı (20.000.002 / 20.000.002, %100). Grafik nettir ve "%100" gerçekten gösterilen veriyle uyumludur.

Bulunan ve giderilen iki kusur (bu test sırasında): (1) Sunucu, uzun full taramayı ortak kilidi tutarak yapıyordu; tarama sürerken alan keşfi ve ikinci istek donuyordu. Artık tarama kilit dışında yapılıyor, UI donmuyor. (2) Full modda alanı değiştirince eski grafik ve yanıtıcı "%100" ekranda kalıyordu (yeni tarama arkada sürerken). Artık yeni istek başlar başlamaz grafik soluklaşıp "taranıyor" diyor; kullanıcı bayat sonucu bitmiş sanmıyor.

kafka-viz

Visualize JSON message fields by sampling the topic tail (MVP)

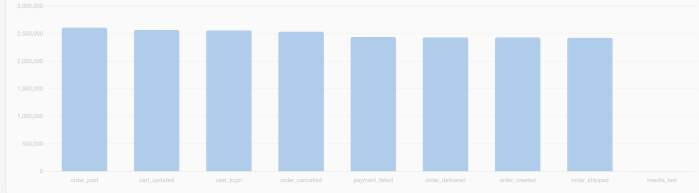
Topic: test-20m View: Value distribution (count) Sampling: Tümü (kesin, yavas) Field: order.currency (text) Time field (optional): ts Sample size: 50000

Tüm veriyi tarayıp kesin sayar; büyük topic'te dakikalar sürebilir. Yalnız değer dağılımı.

Scanning full topic (this can take a while)...

event_type value distribution

Test hakkı sınırlıdır; dakikalar sürebilir...



Alan `order.currency` 'ye çevrildiği an: grafik soluk, meta "taranıyor"; eski "%100" gösterilmiyor (düzeltilen davranış).

kafka-viz

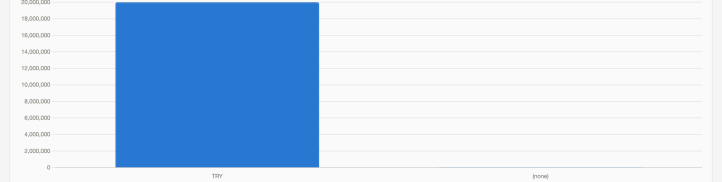
Visualize JSON message fields by sampling the topic tail (MVP)

Topic: test-20m View: Value distribution (count) Sampling: Tümü (kesin, yavas) Field: order.currency (text) Time field (optional): ts Sample size: 50000

Tüm veriyi tarayıp kesin sayar; büyük topic'te dakikalar sürebilir. Yalnız değer dağılımı.

order.currency value distribution

20,000,002 / 20,000,002 messages (100.0%), mode: full/tüm



Tarama bitince grafik doğru alana güncellendi: `order.currency` = neredeyse tümü TRY, kesin sayım (%100).

3. Kurulum: tek adres, tek komut

Tüm parçalar tek bir docker-compose dosyasında toplandı ve önlerine bir yönlendirici (nginx) konuldu. Dışarıya açılan tek port var: 80.

```
cd kafka-ui-demo
docker compose up -d
```

Adres	Açılan
<code>http://localhost/</code>	Kafka UI (fork)
<code>http://localhost/viz/</code>	Grafik aracı (kafka-viz)

Compose içinde 4 servis ayağa kalkar: Kafka (demo broker), Kafka UI (fork imajı `kafka-ui-fix:local`), kafka-viz ve nginx yönlendirici.

Şirket cluster'ına bağlanmak

- **Arayüzden:** dinamik konfigürasyon açık olduğu için (`DYNAMIC_CONFIG_ENABLED=true`) yeni cluster, compose dosyasına dokunmadan arayüz üzerinden eklenebilir.
- Veya compose'daki `KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS` değeri şirket broker adresiyle değiştirilir.

Not: Bu kurulumda kimlik doğrulama (login) bilinçli olarak kapalı; amaç lokal deneme ve demo. Üretime alınırken Kafbat'ın kendi auth desteği (LDAP, OAuth2 vb.) açılacak.

Filtreleme iyileştirmeleri YENİ

Fork'a eklediğimiz üç özelliğin (takılmayan arama, Grafik butonu, tek adres kurulum) ardından, mesaj inceleme ekranına iki filtreleme iyileştirmesi daha ekledik. İkisi de yalnızca arayüz tarafında çalışır, Kafka'ya ek yük bindirmez.

4. Yüklenen sonuçları sıralama

Sorun neydi?

Mesaj listesi yalnızca geldiği sırada gösteriliyordu. En yeni ya da en eski kaydı, en büyük offset'i bulmak için satırları gözle taramak gerekiyordu. Bu, upstream'de yıllardır açık bir istek (issue #1203).

Ne yaptık?

Offset, Partition ve Timestamp sütun başlıklarına tıklamayla sıralama ekledik. İlk tık büyükten küçüğe, ikinci tık küçükten büyüğe sıralar; başlıktaki ok yönü gösterir. Sıralama o an yüklenmiş / taranmış sonuçlar üzerinde çalışır. Canlı akış (Live) modunda kapalıdır, akan verinin doğal sırası bozulmasın diye.

Nasıl kullanılıyor?

Messages ekranında ilgili sütun başlığına (Offset / Partition / Timestamp) tıklamanız yeterli. Tekrar tıklayınca yön değişir.

Sonuç (canlı test)

2 milyon mesajlık test topic'inde 100 kayıtlık bir sette Offset başlığına tıklayınca liste kesin azalan sıraya geçti (338144'ten 330505'e), ikinci tıkta kesin artan sıraya döndü.

Dashboard

demo

Brokers

Topics

Consumers

demo

Topics / test-search

Produce Message

Charts

Overview

Messages

Consumers

Settings

Statistics

ACLs

Newest

Select partitions

Key Serde

Value Serde

Refresh

Search

+ Add Filters

21 ms

28 KB

102 messages consumed

	Offset	Partition	Timestamp	Key	Preview	Value	Preview
+	330505	0	03.07.2026 14:22:00.715			{"event_id":"evt-009411","event_type":"order_deli	
+	330506	0	03.07.2026 14:22:00.715			{"event_id":"evt-007175","event_type":"order_pai	
+	330507	0	03.07.2026 14:22:00.715			{"event_id":"evt-003908","event_type":"cart_upda	
+	330508	0	03.07.2026 14:22:00.715			{"event_id":"evt-002083","event_type":"order_pai	
+	330509	0	03.07.2026 14:22:00.715			{"event_id":"evt-002422","event_type":"order_deli	
+	330510	0	03.07.2026 14:22:00.715			{"event_id":"evt-001926","event_type":"order_can	

Offset başlığına tıklanınca yüklenen mesajlar sıralanır; başlıktaki ok yönü gösterir (burada artan).

5. Tarih aralığı filtresi

Sorun neydi?

Zaman modları tek uçluydu: "belirli bir andan itibaren" veya "belirli bir ana kadar" ayrı ayrı seçiliyordu. "Şu iki zaman arasındaki mesajlar" tek seferde alınamıyordu. Hata ayıklamada en sık ihtiyaç bu ("şu 10 dakikaya bak") ve upstream'de yıllardır açık bir istek (issue #240).

Ne yaptık?

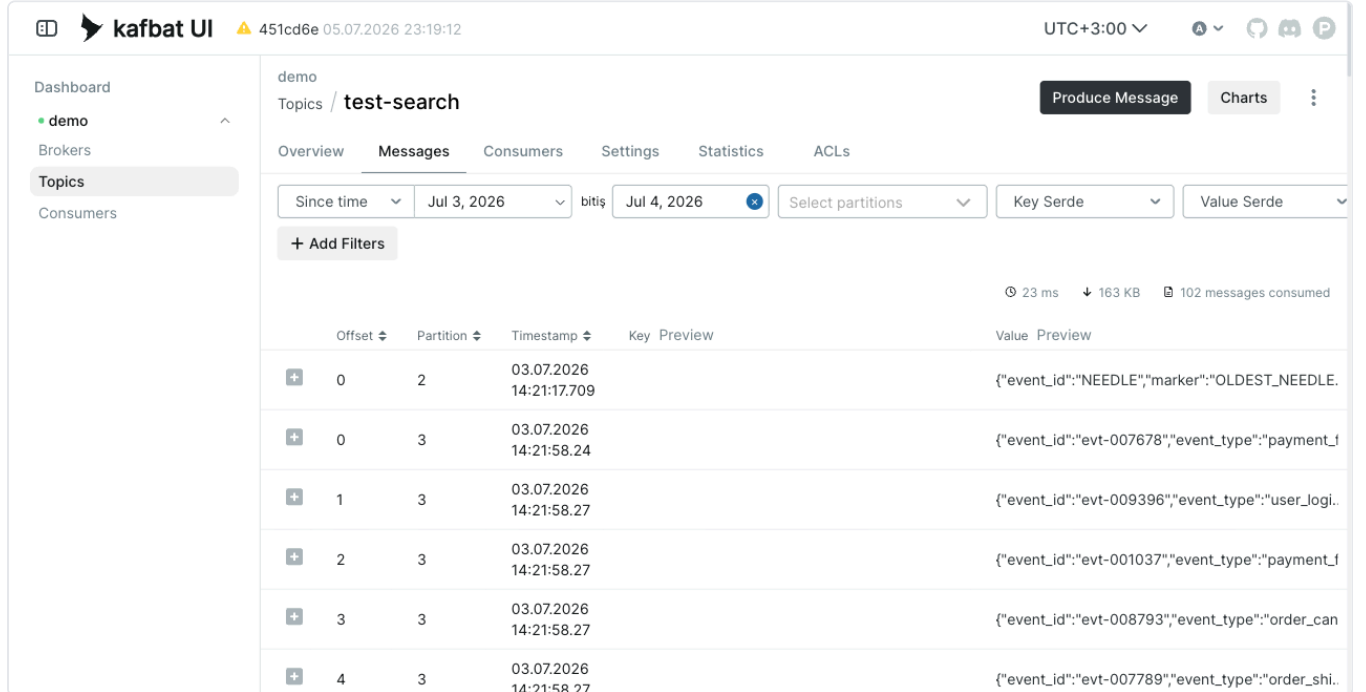
"Since time" (belirli andan itibaren) modunda, başlangıcın yanına isteğe bağlı bir bitiş tarihi seçici ekledik. Başlangıçtan itibaren taranır, gelen mesajlar bitiş anına kadar süzülür. Aralığın sonuna ulaşıncı "Tarih aralığının sonuna ulaşıldı" bilgisi verilir ve ilerleme durur. Süzme tamamen arayüzde yapılır, Kafka'ya ek sorgu yükü binmez.

Nasıl kullanılıyor?

Mod olarak "Since time" seçilir, başlangıç tarihi ve isteğe bağlı bitiş tarihi ("End") girilir. Bitiş boş bırakılırsa üst sınır yoktur (eski davranış aynen korunur).

Sonuç (canlı test)

Başlangıç 3 Temmuz seçiliyken bitiş 4 Temmuz (mesajlardan sonra) yapılıncı tüm mesajlar listelendi. Bitiş 2 Temmuz (mesajlardan önce) yapılıncı liste boşaldı, "Tarih aralığının sonuna ulaşıldı" mesajı çıktı ve Next butonu kapandı.



The screenshot shows the kafbat UI interface for a Kafka topic named 'test-search'. The 'Messages' tab is active, and the 'Since time' filter is set to 'Jul 3, 2026' and the 'End' filter is set to 'Jul 4, 2026'. The table displays 6 messages with their offsets, partitions, timestamps, keys, and previews.

Offset	Partition	Timestamp	Key	Preview
0	2	03.07.2026 14:21:17.709		{ "event_id": "NEEDLE", "marker": "OLDEST_NEEDLE..." }
0	3	03.07.2026 14:21:58.24		{ "event_id": "evt-007678", "event_type": "payment_f..." }
1	3	03.07.2026 14:21:58.27		{ "event_id": "evt-009396", "event_type": "user_logi..." }
2	3	03.07.2026 14:21:58.27		{ "event_id": "evt-001037", "event_type": "payment_f..." }
3	3	03.07.2026 14:21:58.27		{ "event_id": "evt-008793", "event_type": "order_can..." }
4	3	03.07.2026 14:21:58.27		{ "event_id": "evt-007789", "event_type": "order_shi..." }

"Since time" modunda başlangıç ve bitiş tarih seçicileri; aralık içindeki mesajlar listeleniyor.

kafbat UI

451cd6e 05.07.2026 23:19:12

UTC+3:00

Dashboard

demo

Brokers

Topics

Consumers

demo

Topics / test-search

Produce Message

Charts

Overview

Messages

Consumers

Settings

Statistics

ACLs

Since time

Jul 3, 2026

bitiş

Jul 2, 2026

Select partitions

Key Serde

Value Serde

+ Add Filters

23 ms

163 KB

102 messages consumed

Offset

Partition

Timestamp

Key Preview

Value Preview

Tarih aralığının sonuna ulaşıldı.

Next →

Bitiş, yüklenen mesajlardan öncesine ayarlanınca aralık boşalır: "Tarih aralığının sonuna ulaşıldı" ve Next kapalı.

Not: Her iki iyileştirme de o an yüklenmiş / taranmış sonuçlar üzerinde çalışır, tüm topic'i sunucuda yeniden indekslemez. Bu bilinçli bir tercih: düşük risk, hızlı ve Kafka'ya yük bindirmeyen bir çözüm.

Ek düzeltme: Metrics sekmesi artık çalışıyor YENİ

Sorun neydi?

Bir broker'ın **Metrics** sekmesine basınca ekran boş kalıyor ve tarayıcı konsolunda `Unexpected end of JSON input` hatası çıkıyordu. Sebep tek kelimeyle eksik ayar: kurulumda Kafka'ya bir metrik kaynağı (JMX) hiç tanımlanmamıştı. Kaynak olmayınca sunucu bu sekmeye boş bir cevap dönüyor, arayüz de o boş cevabı okumaya çalışıp hata veriyordu. Kafka'nın kendisinde, verinizde veya diğer ekranlarda bir sorun yoktu; etkilenen tek yer bu sekmeydi.

Ne yaptık?

- Kafka broker'ında **JMX'i açtık** (metriklerin okunduğu standart Kafka arayüzü, port 9101).
- Kafka UI'a metrikleri bu porttan okumasını söyledik (`METRICS_TYPE: JMX`).
- Aynı düzenlemede broker'a **kalıcı veri alanı (volume)** ekledik; böylece bundan sonra yeniden başlatma topic'leri silmez.

Artık Metrics sekmesi broker ve JVM metriklerini (istek oranları, kuyruk boyutları, bellek vb.) gerçek değerlerle getiriyor ve konsol hatası kalmadı.

The screenshot shows the kafbat UI interface. On the left is a sidebar with 'Dashboard', 'demo', 'Brokers', 'Topics', and 'Consumers'. The main area is titled 'demo Brokers / Broker 1'. Below this is a table with four columns: 'Segment Size' (0 Bytes), 'Segment Count', 'Port' (9092), and 'Host' (demo-kafka). Underneath the table are three tabs: 'Log directories', 'Configs', and 'Metrics'. The 'Metrics' tab is active, displaying a JSON array of metrics. The JSON structure is as follows:

```
{
  "metrics": [
    {
      "name": "kafka_server_DelayedOperationPurgatory_Value",
      "labels": {
        "delayedOperation": "AlterAcls",
        "name": "NumDelayedOperations"
      },
      "value": 0
    },
    {
      "name": "kafka_server_DelayedOperationPurgatory_Value",
      "labels": {
        "delayedOperation": "AlterAcls",
        "name": "PurgatorySize"
      },
      "value": 0
    },
    {
      "name": "kafka_server_DelayedOperationPurgatory_Value",
      "labels": {
        "delayedOperation": "DeleteRecords",
        "name": "NumDelayedOperations"
      },
      "value": 0
    },
    {
      "name": "kafka_server_DelayedOperationPurgatory_Value",
      "labels": {
        "delayedOperation": "DeleteRecords",
        "name": "PurgatorySize"
      },
      "value": 0
    }
  ]
}
```

Broker 1 > Metrics: JMX üzerinden gelen gerçek metrikler listeleniyor (ör. `kafka_server_...`); tarayıcı konsolunda hata yok. Otomatik test (Playwright) 0 konsol hatası ve veri render ile geçti.

Teknik künye

Taban	Kafbat UI (github.com/kafbat/kafka-ui), Apache-2.0 lisans, upstream commit d0e3a7e üzerine.
Fork dalı	<code>fix/search-polling-budget</code> (arama bütçesi + Grafik butonu).
Grafik aracı	kafka-viz: Python (Flask) + Chart.js, bağımsız küçük servis, salt okur. Modlar: son N / yayılmış / tümü. Yönlendirici (nginx) full tarama için uzun timeout ile ayarlı.
Test ortamı	test-20m topic'i: 20.000.002 sentetik JSON mesaj, 6 partition, tek broker.
Doğrulama	Bu dokümandaki tüm ekran görüntüleri 03.07.2026'da çalışan ortamdan alındı.

Sorular için: Kadir (kadiraycareer@gmail.com)